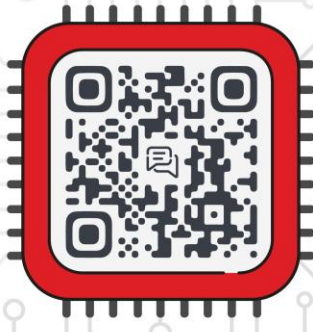




عملي

8



8



960 sp

كلية الهندسة المعلوماتية

السنة الثالثة

التوارد (5) / المعالجات

ذات الإصدارات المتعددة

عملي مشترك

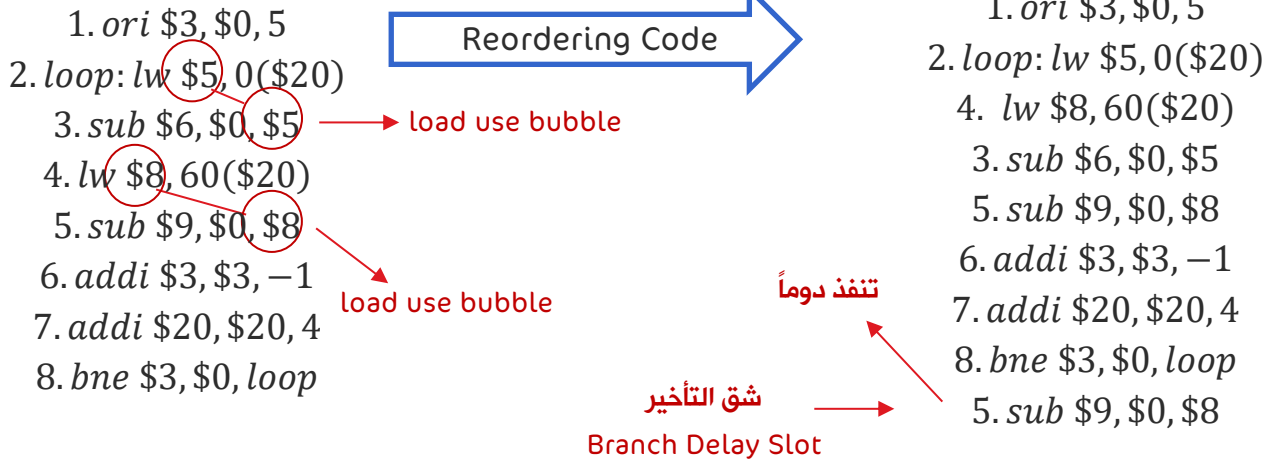


RB Informatique; 2025/12/22

بنیان الحواسيب 2

٤ تمرين (أعطال التحكم):

بفرض لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، وبفرض أن الأعطال البنيوية محلولة، ومع وجود دائرة إحالة، أعد ترتيب البرنامج للإلغاء الفقاعات الموجودة بفرض أن التفرع يكتمل في المرحلة D وبفرض استخدام تقنية التفرع المؤخر Delayed Branch لمعالجة أعطال التحكم:



الشرح: أولاً لدينا ترابط بين التعليمة 1 و 2 بالسجل الخامس هذه تعليمة load وبعدها تعليمة مرتبطة بها إذاً هذا load use Hazard والتعليمة الرابعة والخامسة نفس الحالة، ولدينا فقاعة عند التفرع ونحن نعلم أن هذه الحالة لا يمكن حلها بدارة إحالة فقط بل تحتاج إلى دارة إحالة وفقاعة معاً في هذا التمرين نريد اللجوء إلى إعادة ترتيب البرنامج حتى نتخلص من الفقاعات باستخدام Delayed Branch.

Delayed Branch: في هذه الحالة نحن نخبر البرنامج أن التعليمة التي تلي التفرع نفذها دائماً.

في الحل قمنا بنقل أحد تعليمات sub إلى بعد التفرع والثانية إلى بعد ال load الغير مرتبطة بها وبهذه الحالة تجنبنا حدوث كل الفقاعات وتعليمة sub تنفذها دائماً

التنبؤ: له نوعان إما تنبؤ ستاتيكي أو ديناميكي

الستاتيكي:

- always Take دائماً نتنبأ أنه صحيح
- Always not taken دائماً نتنبأ أنه خطأ

تمرين:

احسب دقة التنبؤ prediction accuracy لكل من تسلسلي التفرعات التاليين:

(a) T, T, N, T, T, T

(b) N, N, T, T, T, T

وذلك في الحالات التالية: (1) التنبؤ ستاتيكي (أ) always taken (ب) always not taken

الحل: (أ) always taken:

a) T, T, N, T, T, T

$$T, T, T, T, T, T \Rightarrow accuracy = \frac{5}{6} = 83.3\%$$

b) N, N, T, T, T, T

$$T, T, T, T, T, T \Rightarrow accuracy = \frac{4}{6} = 66.6\%$$

الحل: (ب) always not taken:

$$a) T, T, N, T, T, T \Rightarrow N, N, N, N, N, N \Rightarrow accuracy = \frac{1}{6} = 16.6\%$$

$$b) N, N, T, T, T, T \Rightarrow N, N, N, N, N, N \Rightarrow accuracy = \frac{2}{6} = 33.3\%$$

شرح الحل: (أ) a) T, T, N, T, T, T التعليمة T, T, T, T, T, T ← التنبؤ

التنبؤ كله صحيح ما عدا التعليمة الثالثة وتكون الدقة $accuracy = \frac{5}{6} = 83.3\%$ وباقي التعليمات بنفس الطريقة

الالتنبؤ الديناميكي: 1bit, 2bit

التنبؤ على 1 بت يكون حسب آخر حالة إذا كانت آخر حالة Taken يكون التنبؤ Taken وإذا كانت آخر حالة not taken يكون التنبؤ not taken

تمتعة التمرين 2-) في حالة التنبؤ الديناميكي باستخدام 1 بت bit predictor بفرض أن التنبؤ عند البدء هو T.

الحل: يكون التنبؤ آخر حالة تفرع فعلية T/N:

a) T, T, N, T, T, T التعليمات



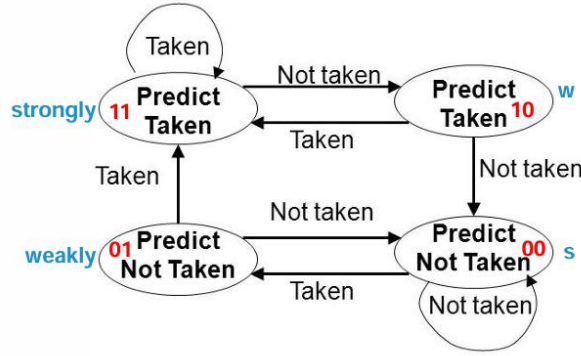
التنبؤ T, T, T, N, T, T

$$accuracy = \frac{4}{6} = 66.6\%$$

b) N, N, T, T, T, T
 T, N, N, T, T, T

$$accuracy = \frac{4}{6} = 66.6\%$$

التنبؤ الديناميكي باستخدام 2bit يستند هذا التنبؤ على مخطط الحالات التالي وهو ثابت ويجب حفظه



تنمية التمرين: 2- في حالة التنبؤ الديناميكي باستخدام 2 بت بفرض أن التنبؤ عند البدء هو sT

a) T, T, N, T, T, T التعليمات

التنبؤ sT, sT, sT, wT, sT, sT

$$accuracy = \frac{5}{6} = 83.3\%$$

b) N, N, T, T, T, T
 sT, wT, sN, wN, sT, sT

$$accuracy = \frac{2}{6} = 33.3\%$$

ملاحظة : يلزم إضافة عتاد لتفريغ قناة التوارد flush من التعليمات التالية لتعليلة التفرع التي بدأت بعد تنبؤ خاطئ



المعالجات ذات الإصدار المتعدد Multiple Issue Processors:

في بنیان الحواسيب، كان الهدف دائماً هو زيادة السرعة وتحسين الأداء، وتعرفنا على مفهوم التوارد Pipelining حيث تنفذ التعليمات بشكل متداخل، ووجدنا أن الحالة المثالية أوصلتنا إلى $CPI \approx 1$.

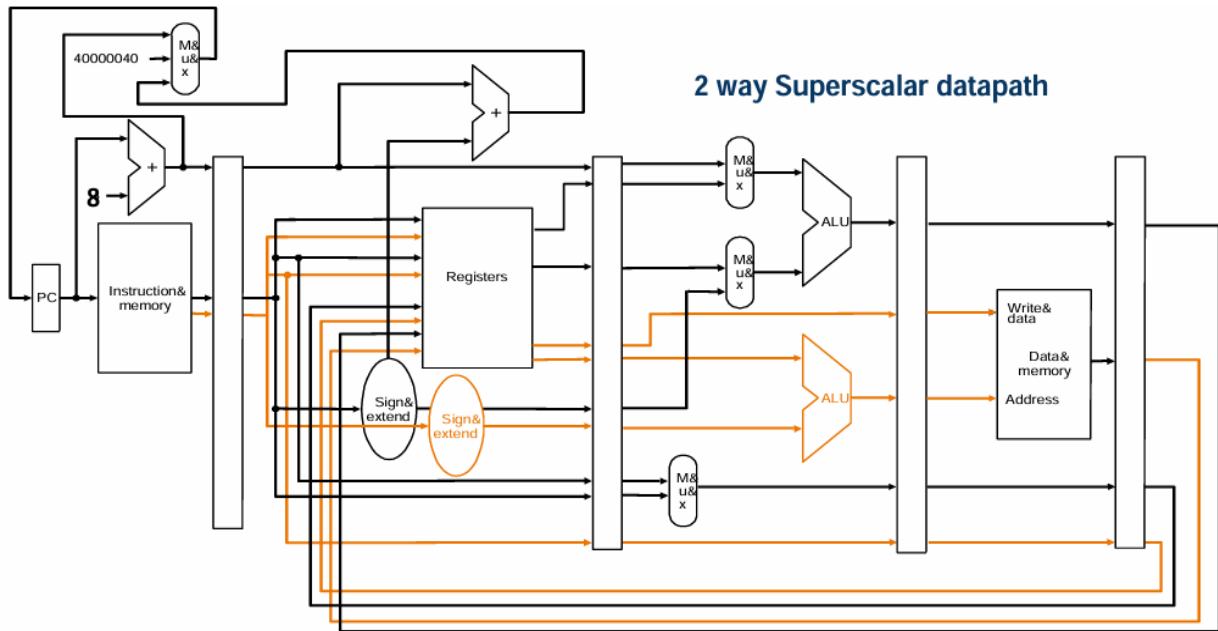
في المعالجات متعددة الإصدارات Multiple Issue Processors بدلاً من أن يرسل المعالج تعليمة واحدة فقط في كل دورة ساعة، فإنه يصدر تعليمتين أو أكثر في وقت واحد، الهدف كسر حاجز انتاج تعليمة واحدة كل نبضة ساعة (الوصول إلى $CPI < 1$)

الإصدار يعني عملية إرسال التعليمات إلى وحدة التنفيذ، وإما أن يكون الإصدارات:

- ستاتيكي: المترجم Compiler هو الذي يقرر أي التعليمات يمكن تنفيذها معاً (مثل معالجات VLIW Very Long Instructions Word Processor) ويتم ذلك قبل وقت التشغيل runtime
- ديناميكي: المعالج (العتاد Hardware) هو الذي يقرر أي التعليمات يمكن تنفيذها معاً ويكون ذلك أثناء التشغيل (Superscalar)

المعالجات ذات البنية السلمي الفائق Superscalar:

الشكل التالي لمعالج ذي بنية سلمية فائق بقناتين two-way superscalar يمكن جلب تعليمتين معاً ثم تنفيذهما وبعدها كتابة النتائج في الواجهة. ويمكن أن يضم أكثر من قناتين. (الرسمه للاطلاع)



أ- بفرض أن لدينا معالج ذو بنیان فائق two-ways Superscalar تمر فيه التعليمات بأربعة مراحل F D E W، وباعتبار الحالة المثالية (No Hazards) احسب CPI من أجل 10 تعليمة، ثم من أجل 5000 تعليمة.

الحل:

- 1 2 3 4 5 6 7 8
1. F D E W
 2. F D E W
 3. F D E W
 4. F D E W
 5. F D E W
 6. F D E W
 7. F D E W
 8. F D E W
 9. F D E W
 10. F D E W

$$CPI = \frac{CC}{IC} = \frac{8}{10} = 0.8 < 1$$

عدد الأدوار الكلي CC = عدد أدوار ملء القناة (عمق القناة = 4) + $\frac{\text{عدد الكليّة التعليمات IC} - \text{عدد القنوات}}{\text{عدد القنوات} (= 2)}$

$$CC = 4 + \frac{5000 - 2}{2} \Rightarrow CC = 2503 \text{ cycle}$$

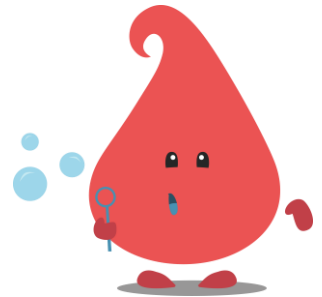
$$CPI_{5000} = \frac{CC}{IC} = \frac{2503}{5000} \approx 0.5$$

كلما زاد عدد التعليمات تقترب قيمة CPI أكثر من القيمة المثالية 0.5 (مقلوب n-way) معلومة: عند حدوث خطأ في التنبؤ سيقوم بعمل Flush أي تفريغ للقنوات بشكل كامل.

ب- بفرض أن التعليمة الثانية كانت تعليمة تفريغ، وأن التفريغ يكتمل في المرحلة E وأن معالجة الأعطال التحكمية تتم باستخدام التنبؤ، ما عدد التعليمات التي سيتم تفريغها (Flush) عند حدوث خطأ في التنبؤ، بفرض عدم وجود أي أعطال أخرى؟

يتم تفريغ التعليمات الأربعة التالية لتعليمة التفريغ!

1. F D E W
2. F D E W
3. F D E W
4. F D E W
5. F D E W
6. F D E W
7. F D E W
8. F D E W
9. F D E W
10. F D E W



أنواع الترابطات في حالة الإصدار المتعدد Multiple Issuing:

RAW(Read After Write): تحدث المشكلة في حالة كون تعليمة تحتاج نتيجة تعليمة سابقة لم تكتمل بعد. يمكن معالجتها بإضافة دالة إحالة، مع وجود تأخير زمني stall في حالة load use، ويمكن كذلك بإعادة ترتيب البرنامج.

ترابط حقيقي true dependence $add \$1, \$2, \$3$

$addi \$4, \$1, 50$

WAR(Write After Read): تحدث المشكلة في حالة أن التعليمة تحاول الكتابة في السجل بينما تعليمة سابقة لم تنته من الكتابة فيه بعد.

ترابط عكسي anti-dependence $or \$8, \$2, \$3$

$add \$2, \$5, \$6$

WAW(Write After Write): تحدث المشكلة في حالة أن التعليمة تحاول تغيير قيمة سجل بينما تعليمة سابقة لا تزال بحاجة لقراءته.

ترابط الإخراج output dependence $or \$1, \$2, \$3$

$add \$1, \$4, \$5$

يمكن التخلص من "false dependence" WAW & WAR بإعادة تسمية السجلات "Register Renaming"

🔑 **تصيرين:**

بفرض لدينا جزء البرنامج التالي:

1. $sub R_1, R_5, R_2$

2. $mult R_5, R_1, R_2$

3. $add R_6, R_1, R_3$

4. $ST R_5, 0(R_6)$

5. $div R_6, R_1, R_5$

6. $beq R_6, R_5, L_1$

1. حدد الترابطات من الأنواع: RAW-WAW-WAR

الترابطات من نوع RAW

$R(1,2), R1(1,3), R1(1,5), R5(2,4), R5(2,5), R5(2,6), R6(3,4), R6(5,6)$

الترابطات من نوع WAR

$R5(1,2), R6(4,5)$

الترابطات من نوع WAW

$R6(3,5)$

2. أعد تسمية السجلات بفرض أن السجلات المتاحة هي من $T_6 \leftarrow T_1$
وما هي الترابطات التي يتم حلها بهذه التقنية؟

تم حل مشكلة الترابطات WAW, WAR

1. *sub* R_1, R_5, R_2
2. *mult* T_1, R_1, R_2
3. *add* R_6, R_1, R_3
4. *ST* $T_1, 0(R_6)$
5. *div* T_2, R_1, T_1
6. *beq* T_2, T_1, L_1

شرح الحل:

الترابطات من النوع (RAW) Read After Write

بالسجل R_1 $R_1(1,3), R_1(1,5), R_1(1,2)$

بالسجل R_5 $R_5(2,5), R_5(2,6), R_5(2,4)$

بالسجل R_6 $R_6(5,6), R_6(3,4)$

الترابطات من النوع (WAR) Write After Read

بالسجل R_5 $R_5(1,2)$

بالسجل R_6 $R_6(4,5)$

الترابطات من النوع (WAW) Write After Write

بالسجل R_6 $R_6(3,5)$



ملاحظات:

حل إعادة تسمية السجلات يحل مشاكل الترابطات من النوع WAR / WAW
وعندما أعيد تسمية سجل انتبه لأن أعيد تسميته لكافة التعليمات المذكورة فيها.

تصيرين:

بفرض البرنامج التالي بلغة التجميع MIPS وبفرض لدينا معالج بإصدار مزدوج Two Issue حيث يمكن تنفيذ تعليمة
ALU/Branch على التوازي مع تعليمات الذاكرة *lw*, *sw*
أعد ترتيب البرنامج من أجل أفضل أداء ممكن بدون فك الحلقة وبفرض وجود دائرة إحالة.

loop:

1. *beq \$s₀, \$zero, end*
2. *lw \$t₀, 0(\$s₁)*
3. *lw \$t₁, 4(\$s₁)*
4. *add \$t₁, \$t₀, t₁*
5. *addi \$t₁, \$t₁, 5*
6. *sw \$t₁, 0(\$s₂)*
7. *addi \$s₁, \$s₁, 8*
8. *addi \$s₂, \$s₂, 4*
9. *addi \$s₀, \$s₀, -1*
10. *j loop*

end:

Cycle	ALU/Branch	Lw/Sw
1	<i>beq \$s₀, \$zero, end</i>	<i>lw \$t₀, 0(\$s₁)</i>
2	<i>addi \$s₁, \$s₁, 8</i>	<i>lw \$t₁, 4(\$s₁)</i>
3	<i>addi \$s₀, \$s₀, -1</i>	<i>NOP</i>
4	<i>add \$t₁, \$t₀, t₁</i>	<i>NOP</i>
5	<i>addi \$t₁, \$t₁, 5</i>	<i>NOP</i>
6	<i>addi \$s₂, \$s₂, 4</i>	<i>sw \$t₁, 0(\$s₂)</i>
7	<i>j loop</i>	<i>NOP</i>

الفكرة الأساسية:

تحسين أداء حلقة مكتوبة بلغة MIPS على معالج Two Issue أي تنفيذ تعليمتين بنفس الدور

يمكن تنفيذ تعليمات ALU/Branch بالتوازي مع lw,sw

شرح البرنامج:

1. قراءة قيمتين من الذاكرة
2. جمع القيمتين
3. نضيف لها 5
4. تخزين النتيجة
5. تحريك المؤشر
6. إنقاص العداد \$s₀
7. تكرار الحلقة

ملاحظة: احتجنا 7 أدوار لتنفيذ نسخة واحدة للحلقة.

⚡ مشكلة الأداء في هذا البرنامج:

كثير من الدورات يضيع فيها وقت، بسبب تعليمات تعتمد على نتائج تعليمات سابقة، أو لأن ALU تكون فارغة (NOP) ولحل هذه المشكلة إعادة الترتيب.



انتهت المحاضرة